2.6 Files

Sue is home for the Christmas holiday when her mother asks her to fix a "computer problem." It turns out that the problem is not the computer itself, but some data their bank has sent them. Instead of e-mailing a list of stock prices in US dollars ($), the entire list is in Euros (€)! Rather than perform the conversion by hand, Sue decides to write a program to do the conversion. This is done by opening the file with the list of Euro prices, performing the conversion to US dollars, and writing the resulting values to another file.

Objectives

By the end of this class, you will be able to:

- Write the code to read data from a file.
- Write the code to write data to a file.
- Perform error checking on file streams.
- Understand the different ways the end-of-file marker can be found.

Prerequisites

Before reading this section, please make sure you are able to:

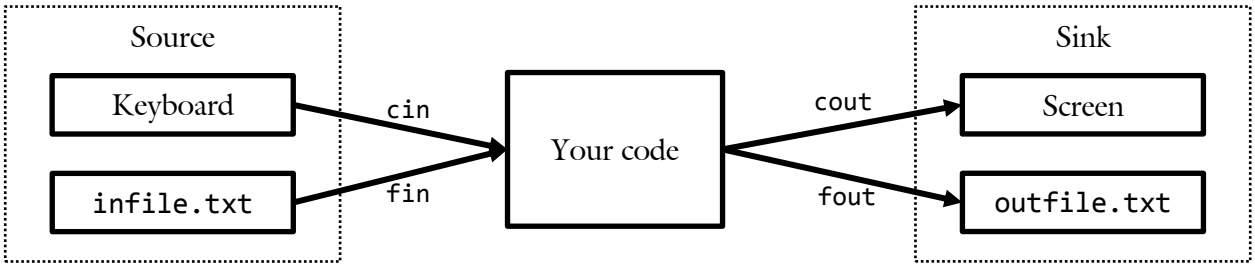
- Create a loop to solve a complex problem (Chapter 2.5).

Overview

After a program ends, all memory of its execution is removed. This fact is particularly unsatisfactory if the program was charged with maintaining the user's valuable data. To overcome this shortcoming, it is necessary to save to and retrieve data from a file.

In many ways, writing data to a file is similar to writing data to the screen. In the file case, however, one needs to specify the target file instead of just using `cout`. In other words, `cout << number << endl;` would put the value of the variable `number` on the screen. If, on the other hand, the variable `fout` corresponded to a file, one could put the value of `number` in the file with `fout << number << endl;`.

Similarly, reading data from a file is similar to accepting user input from a keyboard. Here again, the programmer needs to specify the name of the source file. There is one additional difference, however. When reading data from the keyboard, the programmer can assume there is an infinite amount of data on hand (assuming an infinitely patient user!). Files, on the other hand, are of finite length. At some point, the end will be reached and the program needs to be ready to handle that event.



Writing to a File

When we write text to the screen, we use `cout`. This variable is defined in the `iostream` library and keeps track of where the cursor is on the screen. In other words, as we continue to send data to the screen, the cursor keeps moving to the right or down much the same way a typewriter advances. Sending data to a file is conceptually the same; we need a variable to keep track of the location of the cursor in the file so, as more data is sent to the file, the cursor advances. It follows that we would need a variable very similar to `cout` to do this. However, there is one key difference between writing to a file and writing to the screen: there is only one screen to write to while there may be many files. Therefore, it is necessary to also specify *which* file we are writing to. Consider the following code:

```
#include <fstream>

void writeFile()
{
   // declare the output stream
   ofstream fout;

   // open the file
   fout.open("greeting.txt");

   // write some text
   fout << "Hello world\n";

   // close the stream
   fout.close();
   return;
}
```

Include the FSTREAM library:

All the functions needed to read and write to a file are included in the `fstream` library. It must be included just like `iostream`.

Declare a stream variable:

You must declare a variable associated with the file. Since this is an output stream (writing to a file), use `ofstream`.

Open the stream

This will associate the variable with the file

Insertion operator <<

Write to the file as you would with `cout`

Close the stream

When finished, indicate you are done with the `close()` function

Simple file writing, in other words, consists of several components: the `fstream` library, declaring a stream variable (`fout`), opening the file, streaming data to the file, and closing it when finished.

Sam's Corner

We have previously mentioned that `cout` is a variable. While this is true, it is a special type of variable: an object. An object is a variable that contains both data (position of the cursor on the screen) and functions (think `cout.setw(5)` and `cout.getline(text, 256)`) associated with the data. Objects and the classes that define them are subjects of Object Oriented programming, the topic of CS 165. Don't worry about objects now; this is a topic for next semester.

FSTREAM library

When we were writing programs to display text on the screen, we needed to use the `iostream` library. This library defined two variables (`cout` and `cin`) and the functions associated with them. When writing to a file, we use the `fstream` library. This library contains two data types: `ofstream` and `ifstream`. OFSTREAM stands for Output File STREAM used to send data out of a program and into a file. This is done with:

```
#include <fstream>  // need this line every time a file is used in the program
```

Declaring a stream variable

The next step is to declare a variable that will be used to send data to the file. Note that you may use any variable name you like (as long as it conforms to the Elements of Style guidelines). As a rule, you might want to use `fout` for a stream variable name because it looks and feels like "`cout`." The only difference is that `F` stands for File while `C` stands for Console (or screen). This is done with:

```
ofstream fout;       // declare an output file stream variable.
```

It is possible to have more than one output file stream active at once. A large and advanced program might, for example, write one file for user data, another for a log, and a third to save configuration data. This is not a problem; just have three `ofstream` variables:

```
{
   ofstream foutData;    // For user data
   ofstream foutLog;     // For a log
   ofstream foutConfig;  // For config. data. All 3 can be used at the same time
}
```

Opening a file

After a stream variable is declared, the next step is to connect that variable to a given file. This can be done with a "hard-coded" string (a string literal)…

```
   fout.open("data.txt");       // Always use the same file. We seldom do this.
```

… or it could be done with a variable:

```
{
   char fileName[256];          // string variable to hold the name of the file
   cin >> fileName;             // prompt user for the file name
   fout.open(fileName);         // open the file by referencing the variable.
}
```

It is also possible to both declare a stream variable and associate it with a file in one step:

```
{
   ofstream fout;
   fout.open(fileName);
}
```

```
{
   ofstream fout(fileName);

}
```

Both of the above lines of code do exactly the same thing; it is a matter of convenience which you choose. The final thing to note about opening a file is that, on occasion, there is no file to open. In other words, what would happen when the user attempts to write to a directory that does not exist, to a thumb drive that is full, or to a file where the user lacks the required permission? In each of these cases, an error message will need to be presented to the user. Thus, it is absolutely necessary to check for the error condition and quit the file writing process. Consider the following function:

```
/*********************************************************
 * Write GPA
 * This function will write the user's GPA to a file
 *********************************************************/
bool writeGPA(char fileName[], float gpa)
{
   // declare and open the stream
   ofstream fout(fileName);                 // declare and initialize in one step
   if (fout.fail())                         // check for error with fail()
      return false;                         // indicate to caller that we failed!

   // write the data
   fout << gpa << endl;                     // actually do the work

   // quite
   fout.close();                            // don't forget to close when done
   return true;                             // report success to the caller
}
```

This function takes the filename as a parameter. To call the function, it is nessary to specify a string as the first parameter.

```
{
   writeGPA("myGrade.txt", 3.9);      // first parameter must be a string
}
```

Note how we check for error with the `fail()` function. If we are unable to open the file for writing for any reason (permissions, lack of space, general hardware error, etc.), then `fail()` will return `true`. This will mean that the function `writeGPA()` will be unable to do what it was asked to do: write to a file. We therefore commonly make file functions return Boolean values: `true` corresponds to success and `false` corresponds to failure.

Sam's Corner

By default, OFSTREAM will replace any file of the same name that is being written. This might be what the programmer intended if there is no other file or if data is to be updated. However, it is often necessary to append data onto the end of a file rather than replace it. This can be done by adding another parameter to the output stream declaration:

```
ofstream fout(fileName, ios::app);
```

In this case, the `ios::app` means to append the file rather than overwrite. Other modes include.

Mode	Meaning
`ios::app`	Append output to end of file (EOF)
`ios::ate`	Seek to EOF when the file is opened
`ios::binary`	Open file in binary mode
`ios::in`	Open file for reading. This happens automatically for `ifstream`
`ios::out`	Open file for writing. This happens automatically for `ofstream`
`ios::trunc`	Overwrite existing file instead of truncating. Default for `ofstream`

Streaming data to a file

We use `fout` to send data to a file in exactly the same way we use `cout` to send data to the screen. This means that all tools we had for screen display we also have for file writing. Consider the following code:

```
{
   // configure FOUT for displaying money, just like COUT
   fout.setf(ios::fixed);
   fout.setf(ios::showpoint);
   fout.precision(2);

   // display my budget
   fout << "\t$" << setw(9) << income   << endl;
   fout << "\t$" << setw(9) << spending << endl;
   fout << "\t ---------\n";
   fout << "\t$" << setw(9) << income - spending << endl;
}
```

The above code might be very familiar if `cout` were used instead of `fout`. The only real difference is that this data is sent to a file rather than the screen

Sue's Tips

While it is easy to verify the screen output of a program, it is often inconvenient to verify the file output. As a result, beginner programmers tend to forget details such as putting spaces between numbers. One easy way to work around this tendency is to first write your `writeFile()` function with `couts`. After you have run the program a few times and you are sure of the formatting, turn your `couts` in to `fouts` to send the same data to the file.

Closing the file

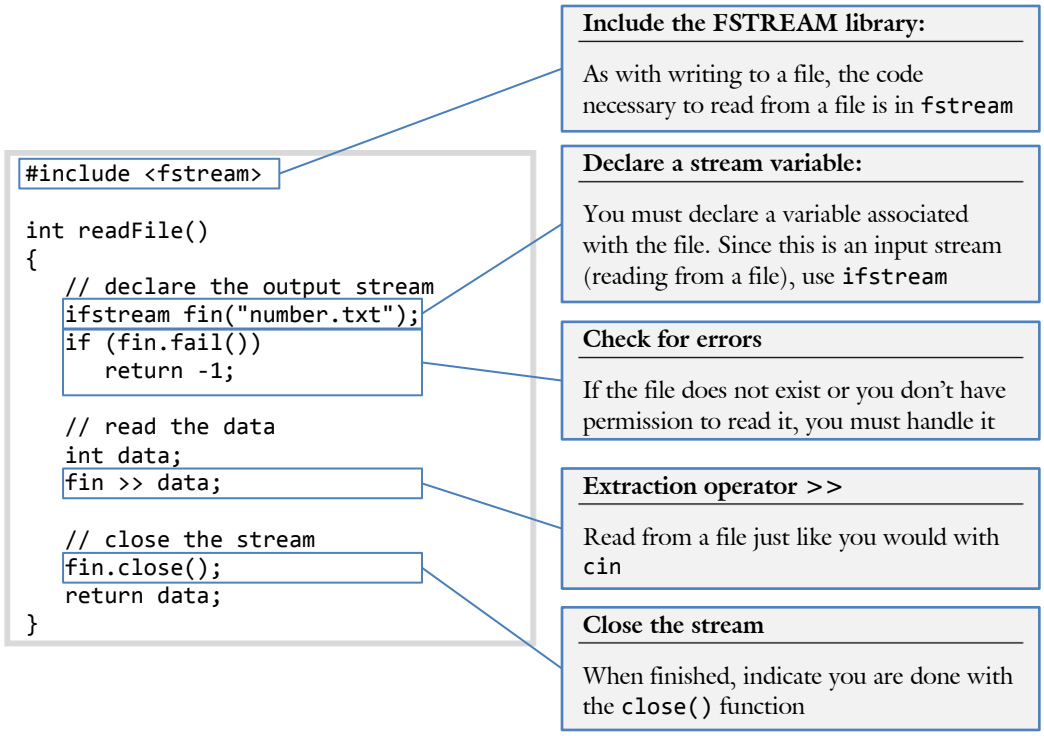
When we are finished writing data to a file, it is important to remember to close the file. On primitive operating systems (think MS-DOS), an un-closed file could never be reopened. Modern operating systems, however, will handle this step for you if you forget. However, it is "good form" to close a file as soon as the last data has been written to it:

```
fout.close();
```

Reading from a File

Just as writing text to a file with `fout` is similar to writing text to the screen with `cout`, reading text from a file has a `cin` equivalent: `fin`. There is, however, one important difference between reading text from the keyboard and reading text from a file. Eventually the end of the file will be reached. It is therefore necessary to make sure logic exists in the program to handle the unexpected end-of-file condition.

As with writing data to a file, several steps are involved: using the FSTREAM library (`#include <fstream>`), declaring the input file stream variable (`ifstream fin;`), checking for errors (`fin.fail()`), using the extraction operator (`fin >> data;`), and closing the file.



FSTREAM library

As with writing to a file, it is necessary to remember to include the FSTREAM library. If this step is skipped, one can expect the following compile error:

```
example.cpp: In function "int readFile()":
example.cpp:6: error: aggregate "std::ifstream fin" has incomplete type and cannot be
defined
```

This cryptic compiler error means that `std::ifstream` is an unknown type. The reason, of course, is that IFSTREAM is defined in `fstream`. Therefore, don't forget:

```
#include <fstream>
```

Declaring a stream variable

Input stream variables are defined in much the same way as output stream variables. The most important difference, of course, is we use `ifstream` for Input File STREAM. Also, like output streams, we can declare and initialize the variable in a single line.

```
{
   ifstream fin;
   fin.open(fileName);
}
```

```
{
   ifstream fin(fileName);

}
```

Again, by convention, it is common to use `fin` for the variable name to emphasize the relationship with `cin`.

Check for errors

As with writing to a file, an essential part of reading from a file includes checking for errors. The same class of errors for writing to a file exists when reading from a file (no permissions, missing directory, general file-system error, etc.). Additionally, the potential exists that there might not be any data in the file to read. In all these cases, we can detect if an error occurred with the `fin.fail()` function call.

```
{
   ifstream fin(fileName);              // attempt to open the file
   if (fin.fail())                      // check for any type of error
   {
      cout << "Unable to open file "    // let the user know!
           << fileName << endl;         // he probably wants to know the file name
      return false;                     // report failure to the user
   }
}
```

Read the data

We write (to the screen or to a file) using the **insertion operator** (<<). Similarly, all read operations are done with the **extraction operator** (>>).

```
{
   float temperature;              // first item is expected to be a number
   char  units[256];               // next item is expected to be text
   fin >> temperature >> units;    // read both just like with cin
}
```

There are two ways we can tell if the read failed for any reason. The first is to check for a read failure. This can be accomplished with another `fin.fail()` function call. The second is to see if the extraction operator itself failed. The following two lines of code are equivalent:

```
{
   int value;
   fin >> value;
   if (!fin.fail())
      cout << "Success!\n";
}
```

```
{
   int value;
   if (fin >> value)
      cout << "Success!\n";

}
```

In other words, the extraction operator (>>) is actually a function call returning `false` when it fails for any reason. One reason may be that the file has been corrupted (or even erased!) during the read. Another may be that there is no more data in the file. Another way to state this last reason is that the "end-of-file" condition may have been met.

Reading to the end of the file

At the end of every file in a file system is a special marker indicating that there is no more data in the file. This can be thought of as the "end of road" marker on a highway. We can ask the file stream if we are at the end of file (EOF) with a function call:

```
{
   if (fin.eof())                          // returns TRUE if we are at the end
      cout << "There is no more data!\n";
}
```

This means that there are two ways to read all the data from a file. The first is to continue looping until the EOF marker is reached. The second is to read until an error has occurred on the read:

EOF	Read Failure
IF the end of the file character is encountered, the EOF flag will be set. You can check for this at any time:	If a read failure occurs, the extraction operator will return `false`. This can be checked on any read.

```
{
   ifstream fin("file.txt");

   while (!fin.eof())
   {
      char text[256];
      fin  >> text;
      cout << text << endl;
   }
   fin.close();
}
```

```
{
   ifstream fin("file.txt");

   char text[256];
   while (fin >> text)
   {
      cout << text << endl;
   }

   fin.close();
}
```

These two methods are not the same. Consider the case when there is a word and a space in the file.

w	o	r	d	space	EOF

In the first case, we will read the word on the first loop and display the text on the screen. On the second iteration, we will go into the body of the loop (because we are not yet at the end of the file: there is still a space left!). When we attempt to read the next word with `fin >> text`, we fail (there is no non-space data in the file after all). In this case, we will not change the value of `text` so the word will be repeated on the screen.

In the second case, we will successfully read the word on the first iteration of the loop. This, of course, will be displayed on the screen in the body of the loop. On the second iteration, we will fail to read (there is no non-space data in the file) so the loop will exit. This means the last word will not be repeated on the screen.

For more details on the aforementioned differences between using the EOF method and the read-failure method of reading from a file, please see Example – End of File on the following page.

Closing the file

As with writing data to a file, it is important to always remember to close the file that was read:

```
fin.close();
```

Sam's Corner

With most operating systems, the failure of a program to close a file is not catastrophic; the operating system will quietly close it for you once the program exits. This is not true on some mobile platforms or older operating systems. An improperly closed file or a file that has not been closed will remain locked and forever unavailable for use. Thus, it is good form to always close a file as soon as reading or writing has been completed.

Filenames

There is one final complication that arises when working with files: the necessity of dealing with filenames. Filenames are c-strings, something we learned about in Chapter 1.2 (page 38) but have done very little with since. The reason for this is that handling c-strings is a bit quirky. We cannot return a c-string from a function as we would any other data-type. Instead, we need to pass it as a parameter.

When passing a c-string, or any other array (which we will learn about in Unit 3), it comes in as pass-by-reference even though we don't have use the '&' operator. Thus the correct way to write a function to prompt the user for a filename is:

```
/*********************************************************
 * GET FILENAME
 * Prompt the user for a filename.
 *********************************************************/
void getFilename(char fileName[])                  // the fileName parameter behaves
{                                                  //    like pass-by-reference
   cout << "What is the name of the file? ";
   cin  >> fileName;                               // text entered here will be sent
}                                                  //    back to the caller
```

If there are some things about this function that you don't understand, don't worry! We will learn more about this on page 245.

Example 2.6 – End of File

Demo

This example will demonstrate how to read all the content of the file using two techniques: either using the EOF method or the Read Failure method.

Problem

Write a program to read all the text out of a file and display the results on the screen. Consider, for example, the following text in a file in `2-6-eof.txt`:

```
I love software development!
```

The output is:

```
Filename? 2-6-eof.txt
Use the EOF method? (y/n): y
'I' 'love' 'software' 'development!' 'development!'
```

Solution

The first solution is to use EOF method.

```
void usingEOF(const char filename[])
{
   // open
   ifstream fin(filename);
   if (fin.fail())
   {
      cout << "Unable to open file " << filename << endl;
      return;
   }

   // get the data and display on the screen
   char text[256];
   // keep reading as long as:
   //   1. not at the end of file
   while (!fin.eof())
   {
      // note that if this fails to read anything (such as when there
      // is nothing but a white space between the file pointer and the
      // end of the file), then text will keep the same value as the
      // previous execution
      fin >> text;
      cout << "'" << text << "' ";
   }
   cout << endl;

   // done
   fin.close();
}
```

The second solution uses the Read Failure method. Everything is the same except the loop:

```
   // keep reading as long as:
   //   1. not at the end of file
   //   2. did not fail to read text into our variable
   //   3. there is nothing else wrong with the file
   while (fin >> text)
      cout << "'" << text << "' ";
```

See Also

The complete solution is available at 2-6-eof.cpp or:

```
/home/cs124/examples/2-6-eof.cpp
```

Example 2.6 – Read Data

Demo

This example will demonstrate how to read a small amount of data from a file. This will include two data types (a string and an integer). All error checking will be performed.

Problem

Consider the following file (`2-6-readData.txt`):

```
Sue 19
```

Read the file and display the results on the screen.

```
What is the filename? 2-6-readData.txt
The user Sue is 19 years old
```

Solution

The man work is performed by the `read()` function, taking a filename as a parameter.

```
bool read(char fileName[])                  // filename we will read from
{
   // open the file for reading
   ifstream fin(fileName);                  // connect to fileName
   if (fin.fail())                          // never forget to check for errors
   {
      cout << "Unable to open file "        // tell the user what happened
           << fileName << endl;
      return false;                         // return and report
   }

   // do the work
   char userName[256];
   int  userAge;
   fin >> userName >> userAge;              // get two pieces of data at once
   if (fin.fail())
   {
      cout << "Unable to read name and age from "
           << fileName << endl;
      return false;
   }

   // user-friendly display
   cout << "The user "                      // display the data
        << userName
        << " is "
        << userAge << " years old\n";

   // all done
   fin.close();                             // don't forget to close the file
   return true;                             // return and report
}
```

Challenge

As a challenge, can you change the above program to accommodate the user's GPA. This will mean that there are three items in the file:

```
Sue 19 3.95
```

See Also

The complete solution is available at 2-6-readData.cpp or:

```
/home/cs124/examples/2-6-readData.cpp
```

Example 2.6 – Read List

Demo

This example will demonstrate how to read large amounts of data from a file. In this case, the file consists of a list of numbers. The program does not know the size of the list at compile time.

Problem

Write a program to sum all the numbers in a file. The numbers are in the following format:

```
34
25
10
43
```

The program will prompt the user for the filename and display the sum:

```
What is the filename: 2-6-readList.txt
The sum is: 112
```

Solution

The function `sumData()` does all the work in this example. It is important to note that the program does not need to remember all the files read from the file. Once the value is added to the `sum` variable, then it can be ignored.

```
int sumData(char fileName[])
{
   // open the file
   ifstream fin(fileName);
   if (fin.fail())
      return 0;                    // some error message

   // read the data
   int data;                       // always need a variable to store the data
   int sum = 0;                    // don.t forget to initialize the variable
   while (fin >> data)             // read: "while there was not an error"
      sum += data;                 // do the work

   // close the stream
   fin.close();
   return sum;
}
```

Challenge

See if you can modify the above program (and the file that it reads from) to work with floating point numbers.

See Also

The complete solution is available at 2-6-readList.cpp or:

```
/home/cs124/examples/2-6-readList.cpp
```

Example 2.6 – Round Trip

Demo

A common scenario is to save data to a file then read the data back again next time the program is run. We call this "round-trip" because the data is preserved through a write/read cycle. This program will demonstrate that process.

Problem

Consider a file consisting of a single floating point number:

```
30.36
```

Write a program to read the file, prompt the user for a value to add to the value, and write the updated value back to the file.

```
Account balance: $30.36
Change: $5.20
New balance: $35.56
```

Solution

First, the program will read the balance from a file. If no balance is found, then return 0.0:

```
float getBalance()
{
   // open the file
   ifstream fin(FILENAME);                // the filename is constant because
   if (fin.fail())                        //     it needs to be same every time
      return 0.0;                         // if no file found, start at $0.00

   // read the data
   float value = 0.0;
   fin >> value;                          // read the old value
   if (fin.fail())                        // if we cannot read this value for any
      return 0.0;                         //     reason, return $0.00

   // close and return the data
   fin.close();
   return value;                          // send the value back to main()
}
```

Then, after the user has been prompted for a new value and the balance has been updated, the new balance is written to the same file.

```
void writeBalance(float balance)
{
   // open the file for writing
   ofstream fout(FILENAME);                // make sure it is the same file as
   …                                       //    we used for getBalance()

   // write the data
   fout.precision(2);                      // format fout for money just like we
   fout.setf(ios::fixed);                  //    would do for cout.
   fout.setf(ios::showpoint);
   fout << balance << endl;                // it is "good form" to end with endl

   …
}
```

See Also

The complete solution is available at 2-6-roundTrip.cpp or:

```
/home/cs124/examples/2-6-roundTrip.cpp
```

Problem 1

What is in the file "data.txt"?

```
void writeData(int n)
{
   ofstream fout;
   fout.open("data.txt");

   for (int i = 0; i < n; i++)
     fout << i * 2 << endl;

   fout.close();
   return;
}

int main()
{
   writeData(4);

   return 0;
}
```

Answer:

Please see page 193 for a hint.

Problem 2

Given the following function:

```
bool writeFile(char fileName[])
{
   ofstream fout;
   fout.open(fileName);

   fout << "Hello World!\n";

   fout.close();

   return true;
}
```

Which would **not** cause the program to fail?

```
writeFile(data.txt);
```

```
writeFile("");
```

```
writeFile(10);
```

```
writeFile("data.txt");
```

Please see page 193 for a hint.

Problem 3

Write a function to put the numbers 1-10 in a file:

- Call the function numbers()
- Pass the file name in as a parameter
- Do error checking

Answer:

Please see page 193 for a hint.

Problem 4

What is the best name for this function?

```
void mystery(char f1[], char f2[])
{
   ofstream fout;
   ifstream fin;

   fin.open(f1);
   fout.open(f2);

   char text[256];
   while (fin.getline(text, 256))
      fout << text << endl;

   fin.close();
   fout.close();

   return;
}
```

Answer:

_____

Please see page 196 for a hint.

Problem 5

Given the following file (`grades.txt`) in the format `<name> <score>`:

```
Jack 83
John 97
Jill 56
Jake 82
Jane 99
```

Write a function to read the data and display the output on the screen. Name the function `read()`.

Please see page 196 for a hint.

Problem 6

Write a function to:

- Open a file and read a number. Display the number to the user.
- Prompt the user for a new number.
- Save that number to the same file and quit.

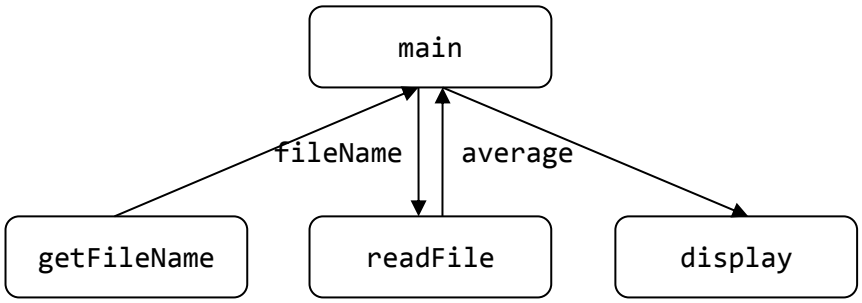
```
The old number is "42". What is the new number?  104
```

Answer:

Please see page 202 for a hint.

Assignment 2.6

Please write a program to read 10 grades from a file and display the average. This will include the functions `getFileName()`, `displayAverage()` and `readFile()`:



getFilename()

This function will prompt the user for the name of the file and return it. The prototype is:

```
void getFileName(char fileName[]);
```

Note that we don't return text the way we do integers or floats. Instead, we pass it as a parameter. We will learn more how this works in Section 3.

readFile()

This function will read the file and return the average score of the ten values. The prototype is:

```
float readFile(char fileName[]);
```

Hint: make sure you only read ten values. If there are more or less in the file, then the function must report an error. Display the following message if there is a problem with the file:

```
Error reading file "grades.txt"
```

display()

This function will display the average score to zero decimals of accuracy (rounded). The prototype is:

```
void display(float average);
```

Example

Consider a file called `grades.txt` (which you can create with emacs) that has the following data in it:

```
90 86 95 76 92 83 100 87 91 88
```

When the program is executed, then the following output is displayed:

```
Please enter the filename: grades.txt
Average Grade: 89%
```

Assignment

The test bed is available at:

```
testBed cs124/assign26 assignment26.cpp
```

Don't forget to submit your assignment with the name "Assignment 26" in the header.

Please see page 201 for a hint.